

编译实验任务说明

2026年4月

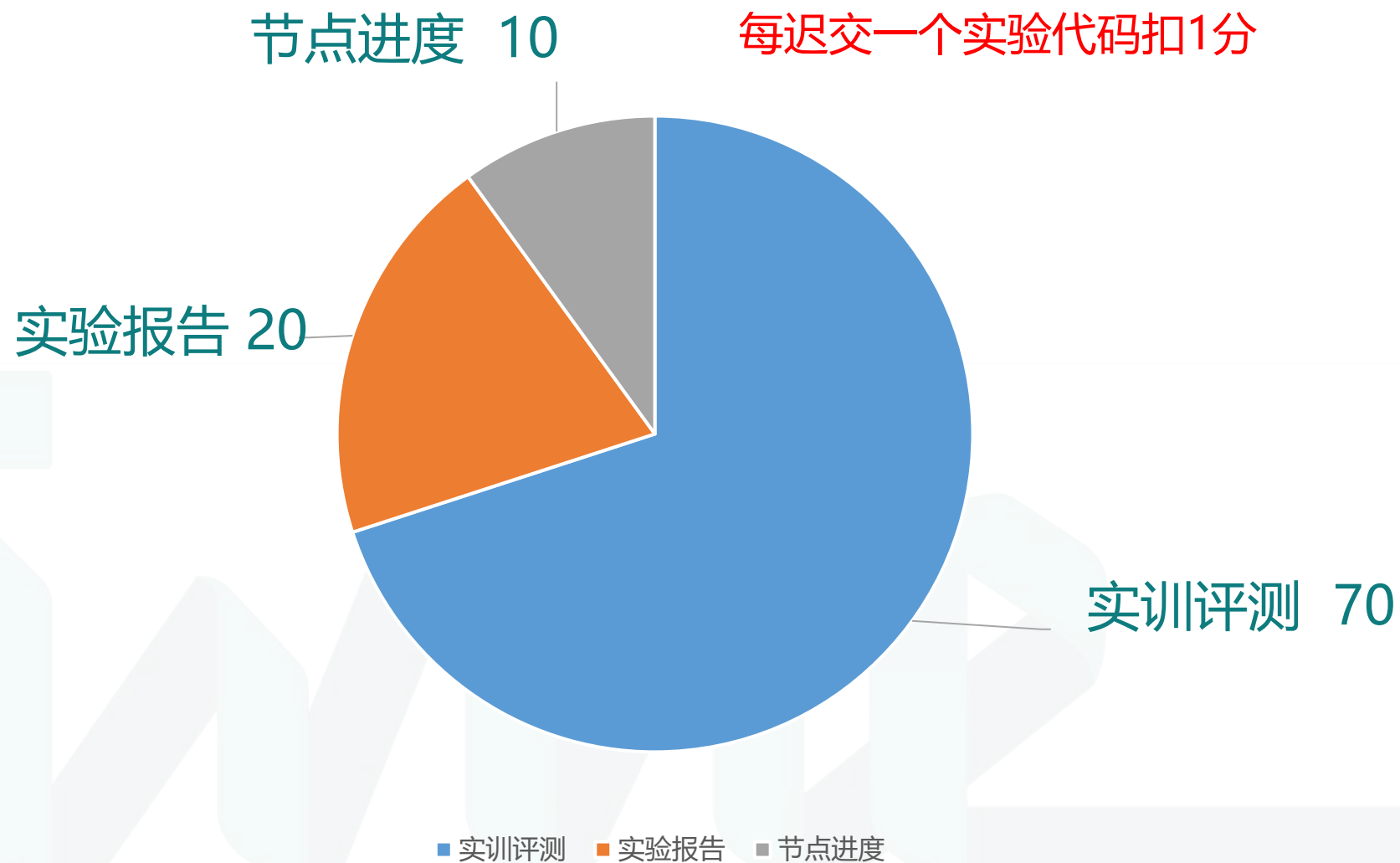
■ 实现一个SYSY语言的编译器，并完成实验报告

■ 途径

- 头歌平台在线6个实验，分阶段完成，自动评测
- 线下实现一个完整编译器，手工评测，工具，语言，目标平台自选
- 特别招募：RUST语言， 鲲鹏架构(aarch64兼容)，手工评测

Rust基金会的白金创始成员之一（谷歌、微软、亚马逊等）
国内唯一的创始成员

成绩评定



评分项目 (分值)	课程目标1 (10分)	课程目标2 (40分)	课程目标3 (30分)	课程目标4 (20分)	最终评定 (100分)
得分					

→ 附件附件补充说明

一、课程说明

本人郑重承诺本报告内容，均来源于本人独立完成的，不存在抄袭、剽窃、复制、拼凑等侵权行为，且内容真实、准确。如发生侵权行为，本人愿意承担一切法律责任。本报告书经本人签字确认后，即成为本人学术成果的重要组成部分，并作为本人学术诚信的重要依据。

课程负责人：XXX 电话：13811111111

二、课程说明的学术评价（教师填写）

该报告内容详实，数据准确，结论合理，具有较高的学术价值。报告结构清晰，逻辑严密，语言表达流畅，符合学术规范。报告内容真实、准确，不存在抄袭、剽窃、复制、拼凑等侵权行为。报告内容经本人签字确认后，即成为本人学术成果的重要组成部分，并作为本人学术诚信的重要依据。

三、课程说明的评价（教师填写）

评价项目	课程目标1 (10分)	课程目标2 (40分)	课程目标3 (30分)	课程目标4 (20分)	总分 (100分)
得分	10	38	28	14	90

教师姓名：XXX

日期：2024.7.10

课程目标	支撑的毕业要求二级指标点
课程目标1	3.3能为计算机复杂工程问题解决方案进行软、硬件系统和应用系统设计，并能在设计中体现创新意识。
课程目标2	4.3能对实验结果进行理论分析，对实验现象进行解释，并能通过信息综合得到合理有效的结论。
课程目标3	5.2能根据计算机领域复杂工程问题解决方案的特定对象特征，选择研究路线，设计实验方案、构建实验系统，并进行实验和正确采集实验数据
课程目标4	10.1能就专业问题，以口头、文稿、图表等方式，准确表达自己的观点，回应质疑，理解与同行和社会公众等不同对象及不同方式交流的差异性

编译原理: <https://www.educoder.net/paths/r72isgp8>

编译原理实验课堂: <https://www.educoder.net/classrooms/VHDQX2CZ>

实验分班:

班级	邀请码
计算机2301	WJDCBT
计算机2302	NKQ3FJ
计算机2303	RDNGXJ
计算机2304	NJ5PLD
计算机2305	8UG9T5
计算机2306	VOURGT

班级	邀请码
计算机2307	KJBZSY
计卓2301	LN75CT
本硕博贯通2301	NZPMJ5

	实验内容	分值	时间分配	说明	截止日期
实验一	工具、语言与平台	15	1W	1-3与1-4任选一	2026/5/10
	1-1 编译工具链的使用	8			
	1-2 SysY语言及运行时库	4			
	1-3 ARM平台	3			
	1-4 RISCv平台	3			
实验二	词法分析	15	1W	任选一个	2026/5/17
	2-1 基于flex的词法分析(C++)	15			
	2-2 基于ANTLR的词法分析(C++)	15			
	2-3 基于ANTLR的词法分析(JAVA)	15			
实验三	语法分析	20	1W	任选一个	2026/5/24
	3-1 基于flex的语法分析(C++)	20			
	3-2 基于ANTLR的语法分析(C++)	20			
实验四	静态语义检查	15	2W		2026/6/7
实验五	中间代码生成	20	2W		2026/6/21
实验六	目标代码生成(基于LLVM)	15	1W	任选一个	2026/6/28
	6-1 arm32代码生成	15			
	6-2 riscv64代码生成	15			

全国大学生计算机系统能力大赛编译系统设计赛



<https://compiler.educg.net>

- 大赛时刻
- 大赛章程与技术方案
- 大赛要求与比赛内容
- 提交方式
- 优秀作品开源
- 技术报告
- 技术支持
- 等级评定标准
- 技术参考
- 常见问题



Sysy语言定义、测试用例和运行时库:

<https://gitlab.eduxiji.net/csc1/nscsc/compiler2025>

两个赛道的优秀作品开源:

<https://compiler.educg.net/#/oldDetail?name=2025全国大学生计算机系统能力大赛编译系统设计赛>

2024决赛作品:

RISCV <https://gitlab.eduxiji.net/educg-group-26173-2487151>

ARM <https://gitlab.eduxiji.net/educg-group-26172-2487152>

■ RUST语言, ARM64后端

□ 教学版

<https://github.com/WyW2357/sysy-compiler>

□ 2025编译赛 电子科技大学

<https://gitlab.eduxiji.net/educg-group-36291-2935673/T202510614205974-2563>

■ RUST语言, RISCV64后端

□ 2024编译赛南开大学

<https://gitlab.eduxiji.net/educg-group-36291-2935673/T202510614205974-2563>

□ 2024编译赛清华大学

<https://gitlab.eduxiji.net/educg-group-26173-2487151/T202410003203847-1935>

■ Win10/11

- 在Microsoft Store搜Ubuntu([u'buntu:/](https://ubuntu.com/))或WSL2, 安装Ubuntu 18/22/24 LTS

- 安装  Visual Studio Code

- 编译基础设施:

C | C++ | java

Flex+Bison | Antlr

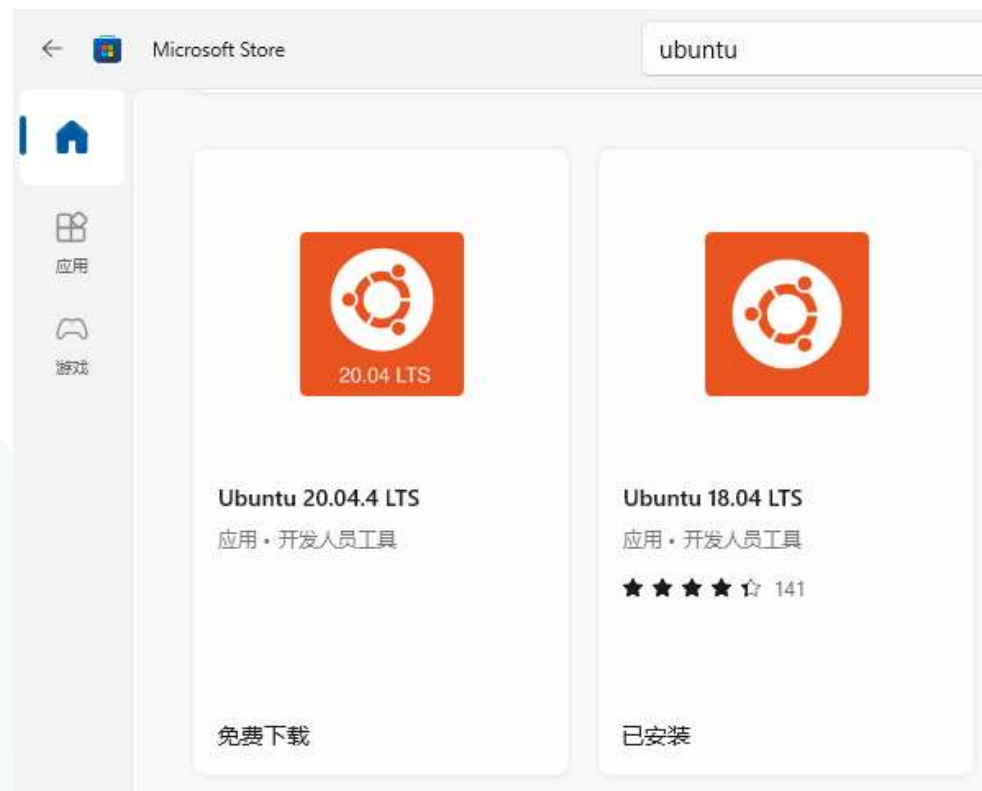
arm-linux-gnueabihf

riscv64-gnu-toolchain

loongarch64-unknown-linux-gnu

qemu-arm/qemu-riscv64/...

LLVM



如何搭建线下平台，准备环境

■ ARM

```
int main() {  
    int arr[7] = {64, 34, 25, 12, 22, 11, 90};  
    return arr[0] + arr[2];  
}
```

ARM®

```
kylin@Lenovokylin:~/src/c$ arm-linux-gnueabi-gcc -S -O2 sort.c -o sort.arm.s  
kylin@Lenovokylin:~/src/c$ arm-linux-gnueabi-gcc sort.arm.s -o sort.arm  
kylin@Lenovokylin:~/src/c$ qemu-arm -L $ARM sort.arm  
kylin@Lenovokylin:~/src/c$  
kylin@Lenovokylin:~/src/c$ echo $?  
89
```

```
kylin@Lenovokylin:~/src/c$ echo $ARM  
/usr/arm-linux-gnueabi/
```

ARM64: aarch64-linux-gnu-gcc, qemu-aarch64

如何搭建线下平台，准备环境

■ RISC-V



```
int main() {  
    int arr[7] = {64, 34, 25, 12, 22, 11, 90};  
    return arr[0] + arr[2];  
}
```

```
kylin@Lenovokylin:~/src/c$ riscv64-linux-gnu-gcc -S -O2 sort1.c -o sort.riscv.s
```

```
kylin@Lenovokylin:~/src/c$ riscv64-linux-gnu-gcc sort.riscv.s -o sort.riscv
```

```
kylin@Lenovokylin:~/src/c$ qemu-riscv64 -L $RISCV sort.riscv
```

```
kylin@Lenovokylin:~/src/c$ echo $?
```

```
89
```

```
kylin@Lenovokylin:~/src/c$ echo $RISCV
```

```
/usr/riscv64-linux-gnu/
```

头歌平台上的交叉编译器是 riscv64-unknown-elf-gcc

如何搭建线下平台，准备环境

■ 龙芯

```
int main() {  
    int arr[] = {64, 34, 25, 12, 22, 11, 90};  
    int n = sizeof(arr)/sizeof(arr[0]);  
    printf("未排序数组: \n");  
    printArray(arr, n);  
    bubbleSort(arr, n);  
    printf("排序后数组: \n");  
    printArray(arr, n);  
    return 0;  
}
```

龙芯中科
LOONGSON TECHNOLOGY

```
kylin@Lenovokylin:~/src/c$ loongarch64-unknown-linux-gnu-gcc -S -O2 sort.c -o sort.loongarch.s  
kylin@Lenovokylin:~/src/c$ loongarch64-unknown-linux-gnu-gcc -static sort.loongarch.s -o sort.loongarch  
kylin@Lenovokylin:~/src/c$ qemu-loongarch64 sort.loongarch  
未排序数组:  
64 34 25 12 22 11 90  
排序后数组:  
11 12 22 25 34 64 90  
kylin@Lenovokylin:~/src/c$
```

■ 所有实验都开放了命令行

头歌服务器:

```
root@educoder:/data/workspace/myshixun# git remote get-url origin  
https://git.educoder.net/prxpafhei/frcwhqxtla20230508210723.git
```

本地:

```
$git clone https://git.educoder.net/prxpafhei/frcwhqxtla20230508210723.git
```

头歌平台演示: <https://www.educoder.net/classrooms/VHDQX2CZ>

实验1~3说明



ANTLR: 词法和语法分析



■ 给定文法

(1) $S \rightarrow AaS$ $\text{SELECT}() = \{ a \}$

(2) $S \rightarrow BbS$ $\text{SELECT}() = \{ b, c \}$

(3) $S \rightarrow d$ $\text{SELECT}() = \{ d \}$

(4) $A \rightarrow a$ $\text{SELECT}() = \{ a \}$

(5) $B \rightarrow \epsilon$ $\text{SELECT}() = \{ b \}$

(6) $B \rightarrow c$ $\text{SELECT}() = \{ c \}$

■ Demo.g4

命名一致

(1) $S \rightarrow AaS$

(2) $S \rightarrow BbS$

(3) $S \rightarrow d$

(4) $A \rightarrow a$

(5) $B \rightarrow \epsilon$

(6) $B \rightarrow c$

```
grammar Demo;
```

```
A : 'a';
```

```
B : 'b';
```

```
C : 'c';
```

```
D : 'd';
```

```
root : s EOF ;
```

```
s : a A s | b B s | D ;
```

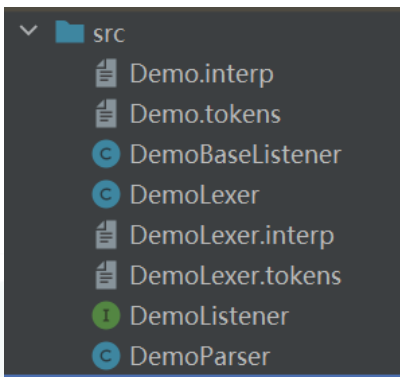
```
a : D;
```

```
b : C | ;
```

- 枚举词法规则(终结符大写)
- 添加新的文法起始符号
- 约定非终结符小写
- 用意义的命名
- 注意空规则的表达

■ Demo.g4 → Demo.java

```
java -cp lib\antlr-4.13.1-complete.jar org.antlr.v4.Tool demo.g4 -o src\
```



```
public class Main {  
  
    public static void main(String[] args) throws IOException {  
  
        final int EOF=-1, A=1, B=2, Whitespace=3, Newline=4;  
  
        String[] tokenNames = { null, "A", "B", "WS", "NewLine" };  
  
        if (args.length < 1) {  
            System.out.println("Source file not given!\n");  
            return;  
        }  
  
        CharStream input = CharStreams.fromFileName(args[0]);  
        DemoLexer lexer = new DemoLexer(input);  
        BufferedTokenStream tokens = new BufferedTokenStream(lexer);  
        tokens.fill();  
  
        for (Token token : tokens.getTokens()) {  
            int token_type = token.getType();  
            if (token_type != -1)  
                System.out.println(token.getText() + " : " + tokenNames[token.getType()]);  
        }  
  
        DemoParser parser = new DemoParser(tokens);  
        ParseTree tree = parser.root(); // begin parsing at init rule  
        System.out.println(tree.toStringTree(parser)); //output parser tree  
    }  
}
```

■ Main.java

```
public class Main {  
  
    public static void main(String[] args) throws IOException {  
  
        CharStream input = CharStreams.fromFileName(args[0]);  
        DemoLexer lexer = new DemoLexer(input);  
        BufferedTokenStream tokens = new BufferedTokenStream(lexer);  
        tokens.fill();  
  
        for (Token token : tokens.getTokens()) {  
            int token_type = token.getType();  
            if (token_type != -1)  
                System.out.println(token.getText() + " : " + tokenNames[token.getType()]);  
        }  
  
        DemoParser parser = new DemoParser(tokens);  
        ParseTree tree = parser.root(); // begin parsing at init rule  
        System.out.println(tree.toStringTree(parser)); //output parser tree  
    }  
}
```

```
import org.antlr.v4.runtime.*;  
import org.antlr.v4.runtime.tree.*;
```

|| 实战演练: ANTLR4

■ DemoParser.java

```
public final RootContext root() throws RecognitionException {
    RootContext _localctx = new RootContext(_ctx, getState());
    enterRule(_localctx, 0, RULE_root);
    try {
        enterOuterAlt(_localctx, 1);
        {
            setState(8);
            s();
            setState(9);
            match(EOF);
        }
    }
}
```

- (1) $S \rightarrow AaS$ $SELECT() = \{ a \}$
- (2) $S \rightarrow BbS$ $SELECT() = \{ b, c \}$
- (3) $S \rightarrow d$ $SELECT() = \{ d \}$

```
public final SContext s() throws RecognitionException {
    case A:
        enterOuterAlt(_localctx, 1);
        {
            setState(11);
            a();
            setState(12);
            match(A);
            setState(13);
            s();
        }
        break;
    case B:
    case C:
        enterOuterAlt(_localctx, 2);
        {
            setState(15);
            b();
            setState(16);
            match(B);
            setState(17);
            s();
        }
        break;
    case D:
```

- 指定生成代码的语言, 指定代码存放位置, ...
- 需安装ANTLR 运行时库
- include头文件, 链接时指定运行时库

```
java -cp "/usr/local/lib/antlr-4.9.3-complete.jar:$CLASSPATH" org.antlr.v4.Tool Sysy.g4 -Dlanguage=C++ -listener -visitor -o src/
```



Flex+bison: 词法和语法分析



■ 给定文法

- 1) $S \rightarrow L = R$
- 2) $S \rightarrow R$
- 3) $L \rightarrow * R$
- 4) $L \rightarrow id$
- 5) $R \rightarrow L$

注：
S: start
L: lval
R: rval
id:ID

项目文件

```
.  
├── tokens.l      # 词法规则  
├── parser.y      # 语法规则  
├── main.cpp      # 主程序入口  
├── flex-bison.sh # 分析器代码生成脚本  
├── MAKEFILE      # 项目构建makefile  
├── gen-graph.sh  # LR自动机图转换脚本  
└── test.txt      # 测试输入文件
```

```
value = *address  
*ptr = data  
simple  
i = j  
i = j = k
```

自底向上语法分析器的自动生成(flex + bison)

■ tokens.l

- 1) $S \rightarrow L = R$
- 2) $S \rightarrow R$
- 3) $L \rightarrow * R$
- 4) $L \rightarrow \text{id}$
- 5) $R \rightarrow L$

S: start
L: lval
R: rval
id:ID

```
%option noyywrap
%option yylineno

%{
#include <string>
#include <iostream>
#include "parser.hpp"
int yycolumn=1;
#define YY_USER_ACTION {...}
%}

ID [a-z_A-Z][a-z_A-Z0-9]*
WHITESPACE [ \t]+

%%
WHITESPACE {
{ID}      { yylval.str = strdup(yytext); return ID;}

[\n]      { yycolumn=1; return NEWLINE;}
.         { return yytext[0];}
```

自底向上语法分析器的自动生成(flex + bison)

■ parser.y

- 1) $S \rightarrow L = R$
- 2) $S \rightarrow R$
- 3) $L \rightarrow *R$
- 4) $L \rightarrow \text{id}$
- 5) $R \rightarrow L$

S: start
L: lval
R: rval
id:ID

```
%define parse.error verbose
%locations
%{
    #include <memory>
    #include <cstring>
    #include <stdarg.h>
    using namespace std;

    extern int yylineno;
    extern int yylex();
    extern void yyerror(const char *s);
    extern void initFileName(char *name);
    char filename[100];
}%

%union {
    char* str;
};
```


自底向上语法分析器的自动生成(flex + bison)

■ parser.y

- 1) $S \rightarrow L = R$
- 2) $S \rightarrow R$
- 3) $L \rightarrow * R$
- 4) $L \rightarrow id$
- 5) $R \rightarrow L$

S: start
L: lval
R: rval
id:ID

```
%token '=' '*'
%token <str> ID NEWLINE
%type <str> lval rval
%start start

%%

start: lval '=' rval {
    | rval { }
    ;

lval: '*' rval { }
    | ID { }
    ;

rval: lval { } ;

%%
```

自底向上语法分析器的自动生成(flex + bison)

■ parser.y

- 1) $S \rightarrow L = R$
- 2) $S \rightarrow R$
- 3) $L \rightarrow *R$
- 4) $L \rightarrow \text{id}$
- 5) $R \rightarrow L$

S: start
L: lval
R: rval
id:ID

```
void initFileName(char *name){
    strcpy(filename, name);
}

void yyerror(const char* fmt) {
    printf("%s: line %d ", filename, yylloc.first_line);
    printf("%s\n", fmt);
}
```

■ main.cpp

- 1) $S \rightarrow L = R$
- 2) $S \rightarrow R$
- 3) $L \rightarrow *R$
- 4) $L \rightarrow \text{id}$
- 5) $R \rightarrow L$

S: start
L: lval
R: rval
id:ID

```
#include <fstream>
#include <iostream>
#include <cassert>
#include <string>
#include <cstring>

extern int yyparse();
extern void initFileName(char *);
extern FILE *yyin;

int main(int argc, char **argv) {
    if (argc < 2) {
        std::cerr << "用法: " << argv[0] << " <文件名>" << std::endl;
        return 1;
    }
}
```

■ main.cpp

- 1) $S \rightarrow L = R$
- 2) $S \rightarrow R$
- 3) $L \rightarrow *R$
- 4) $L \rightarrow id$
- 5) $R \rightarrow L$

S: start
L: lval
R: rval
id:ID

```
FILE* file = fopen(argv[1], "r");
if (!file) {
    std::cerr << "错误: 无法打开文件 " << argv[1] << std::endl;
    return 1;
}

yyin = file;
initFileName(argv[1]);

int result = yyparse();
fclose(file);

if (result == 0) {
    std::cout << "语法分析完成, 未发现错误。" << std::endl;
}
return result;
}
```

■ 分析器代码自动生成

1) $S \rightarrow L = R$
2) $S \rightarrow R$
3) $L \rightarrow * R$
4) $L \rightarrow \text{id}$
5) $R \rightarrow L$

S: start
L: lval
R: rval
id:ID

```
bison -d -v -o parser.cpp --graph=parser.dot parser.y  
flex -o tokens.cpp tokens.l  
dot -Tpng -oparser.png parser.dot
```

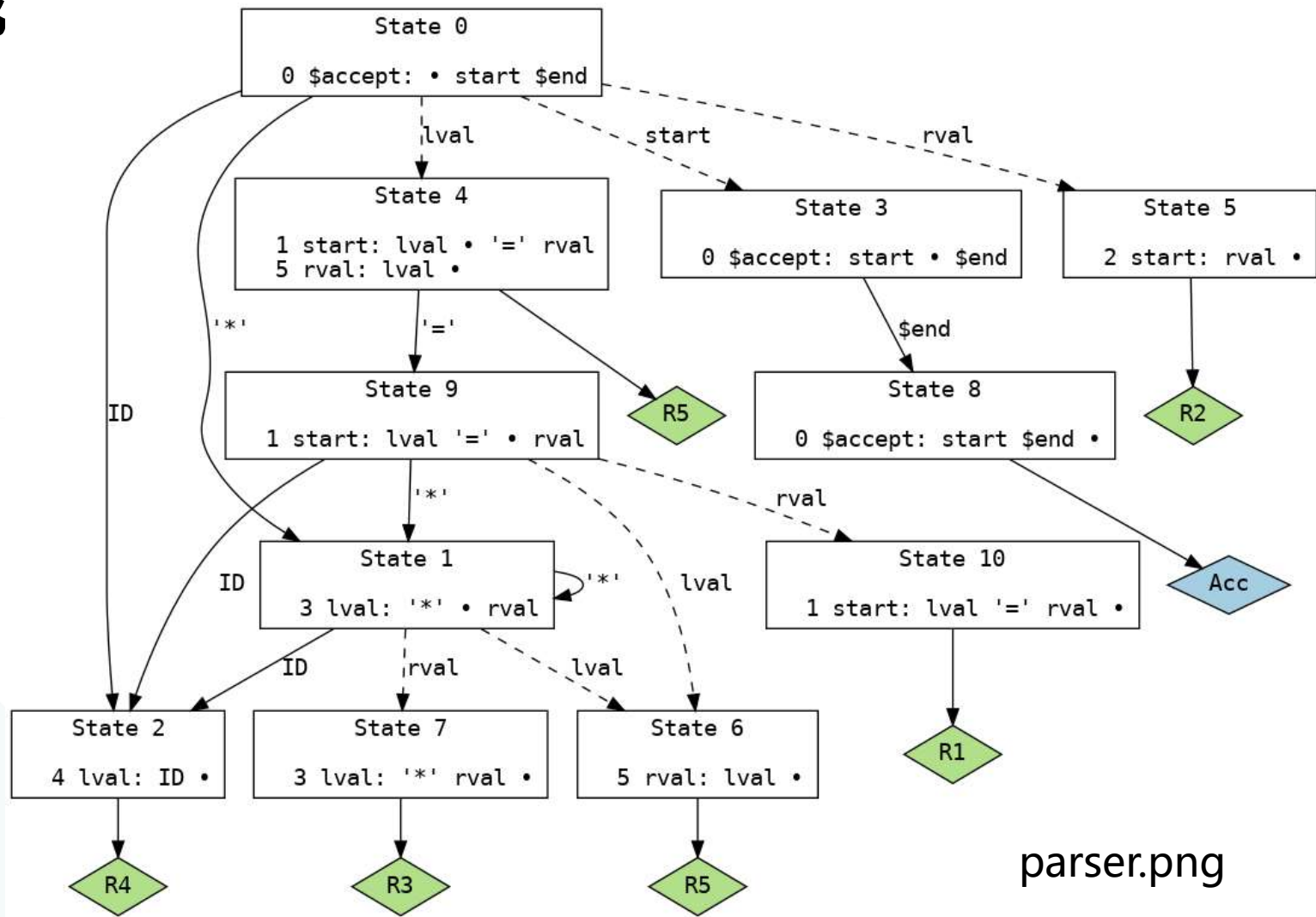
注:

- d 生成头文件
 - v 生成parser.output报告, 说明DFA状态, 移进归约动作
 - graph 生成LR自动机图形描述文件 (文本文件)
 - o 指定生成的词法/语法分析程序文件名
- 可以用dot将图形描述文件转换成SVG或PNG图

■ 分析器代码自动生成

- 1) $S \rightarrow L=R$
- 2) $S \rightarrow R$
- 3) $L \rightarrow *R$
- 4) $L \rightarrow id$
- 5) $R \rightarrow L$

S: start
L: lval
R: rval
id:ID



parser.png

■ 分析器代码自动生成

- 1) $S \rightarrow L = R$
- 2) $S \rightarrow R$
- 3) $L \rightarrow * R$
- 4) $L \rightarrow id$
- 5) $R \rightarrow L$

S: start
L: lval
R: rval
id:ID

./parser test.txt

```
.  
├── Makefile  
├── flex-bison.sh  
├── gen-graph.sh  
├── llvm-cfg.sh  
├── main.cpp  
├── main.o  
├── parser  
├── parser.cpp  
├── parser.dot  
├── parser.hpp  
├── parser.o  
├── parser.output  
├── parser.png  
├── parser.y  
├── test.txt  
├── tokens.cpp  
├── tokens.l  
└── tokens.o
```

三争朝夕 不負時代

SEIZE THE DAY
AND LIVE UP TO THE TIMES

Teamwork Innovation
协作 创新
修德 开拓
Morality Exploration

再见



- <https://www.educoder.net/>
- <https://godbolt.org>
- <https://www.antlr.org>
- <https://www.llvm.org/>